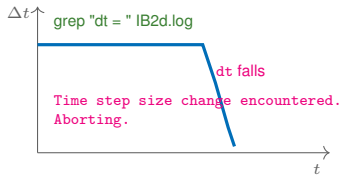
WHAT CAN GO WRONG, CONVERGENCE TESTS AND DEBUGGING

The failures of Part III collected and explained, the 2014 troubleshooting rules checked against the code, a convergence study on the bouncing band, and a way of working when a run misbehaves



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 16

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. recognise the ways an IB simulation fails and read what the program says about it: the CFL abort, a point escaping the box, a parse error, a silent key, a segmentation fault at the first output, a job that the scheduler kills;
2. apply the 2014 troubleshooting rules (ramp the flow, keep the stiffness down, refine where the vortices are, tighten the solver, match every dimensionless number) and know which of their keys and commands still exist in 0.19.0;
3. watch a run while it goes, by the time step in the log, and stop it before it wastes a day;
4. say what convergence, order and stability mean, run the stiff-equation and heat-equation exercises, and carry out a grid-refinement study on an IB example as this chapter does on the bouncing band;
5. work through a misbehaving run in a fixed order, from the smoke test to the debugger, without guessing.

> Chapters 5–15 each ended with a “what can go wrong” slide. This chapter is the union of those, sorted by what you see rather than by where it was found, plus the two things the series had not yet done: a convergence test and a debugging routine.

HOW THIS CHAPTER WAS CHECKED

- Every error message quoted is taken from the IBAMR 0.19.0 source (`IBHierarchyIntegrator.cpp`, `LDataManager.cpp`, `ConstraintIBKinematics.cpp`, `IBStandardInitializer.cpp`) or from a run of this series, with the chapter where it happened; every input key is one the source reads.
- The 2014 tutorial *What can go wrong* (14 slides) is quoted in full and checked line by line: two of its keys, one command and one formula are examined against 0.19.0 (slides 9, 11, 13).
- The 2021 lecture *Stability and convergence* (MATH 481 Lecture 19, 40 slides) supplies the definitions and the two MATLAB exercises, which this series ships with Python versions (`examples/support/matlab-exercises`); its IB2d lab is translated to IBAMR terms.
- The convergence study was run for this chapter: IBAMR's `examples/IB/explicit/ex1` at four grids, 64 to 512 cells, with the shipped structure files for each, and the band's shape compared across them (slides 26, 27).
- The 2019/2020 cluster debugging tips (four slides, UNC's Dogwood) are summarised for Puma; their two inaccuracies are noted.

WHERE TO LOOK THINGS UP

- The messages: `grep -rn "TBOX_ERROR" <IBAMR>/src <IBAMR>/ibtk/src` lists every way IBAMR can stop on purpose, with the text it prints; the ones a user meets are on slides 31 and 32.
- The time step: `IBHierarchyIntegrator.cpp`, `preprocessIntegrateHierarchy` (the dt-change check, keys `error_on_dt_change`, `warn_on_dt_change`) and `postprocessIntegrateHierarchy` (the CFL numbers it logs).
- Refinement: `INSHierarchyIntegrator.cpp`, `vorticity_rel_thresh`, `vorticity_abs_thresh`; solver tolerances: `INSStaggeredHierarchyIntegrator.cpp` (`stokes_solver_type`, `stokes_solver_db`, the PETSc prefix `stokes_`).
- This series: chapter 3 (build and input-file errors), 7 (boundary conditions), 8a and 8b (parameters, dimensionless numbers), 9a–10 (stiffness and time step), 13 (penalty stiffness), 14 (silent keys), 15 (building the upstream examples); IB/01's README has the time-step table.
- Sources: the 2014 *What can go wrong* deck, the 2021 *Stability and convergence* lecture, the 2012 lab ReadMe (slide 5), the 2019 *Debugging tips*.

THE OLDEST ADVICE: WATCH THE TIME STEP

"The output will be dumped into viz_IB2d. There will also be some diagnostic info dumped into the file IB2d.log. You will probably want to keep an eye on the timestep size. I do this via % `grep "dt = " IB2d.log` or % `tail -f IB2d.log | grep "dt = "`. If you see that the timestep size decreases dramatically, this usually means that the computation has gone unstable. Reduce the value of 'DT' in input2d and restart the model. The timestep size may be too small if the program crashes or the results are very noisy as well." (lab ReadMe, 2012, slide 5)

```
# 0.19.0, with enable_logging = TRUE: the same line is still
  there
$ grep "dt = " IB2d.log | head -3
IBHierarchyIntegrator::advanceHierarchy(): time interval =
  [0,0.00025], dt = 0.00025
IBHierarchyIntegrator::advanceHierarchy(): time interval =
  [0.00025,0.0005], dt = 0.00025
IBHierarchyIntegrator::advanceHierarchy(): time interval =
  [0.0005,0.00075], dt = 0.00025
$ grep -c "At end" run.log        # steps done
$ tail -f run.log | grep "Simulation time"
```

Thirteen years on this is still the first thing to do. Three refinements:

- the step is *fixed* at `DT` (or `DT_MAX`) unless the CFL limit `cfl` $\times \Delta x / |u|_{\max}$ is smaller; in 0.19.0 a step that *changes* is not merely a warning (slide 6);
- the log also prints the CFL numbers at the end of every step ("CFL number = ..."), the fluid's and the structure's regrid estimates) when logging is on; a CFL number climbing toward `CFL_MAX` is the instability announcing itself;
- `integral{F}` in the screen output (IB/01, chapter 5) is the total Lagrangian force: for a closed structure it should stay at round-off; a growing value is a structure tearing itself apart.

TIME STEP SIZE CHANGE ENCOUNTERED

```
// src/IB/IBHierarchyIntegrator.cpp, preprocessIntegrateHierarchy() (0.19.0)
if (!skip_check_for_dt_change && (d_error_on_dt_change || d_warn_on_dt_change) &&
    !IBTK::rel_equal_eps(dt, d_dt_previous[0]) && !IBTK::rel_equal_eps(new_time, d_end_time))
{
    if (d_error_on_dt_change)
    {
        TBOX_ERROR(d_object_name << "::preprocessIntegrateHierarchy():  Time step size change encountered.\n"
                                 << "Aborting." << std::endl);
    }
    if (d_warn_on_dt_change)
    {
        pout << d_object_name << "::preprocessIntegrateHierarchy():  WARNING: Time step size change encountered.\n"
```

```
IBHierarchyIntegrator {
   ...
   error_on_dt_change = FALSE   // default TRUE: abort when dt changes
   warn_on_dt_change  = TRUE    // print instead (aliases: error_on_timestep_change, error_on_time_step_change)
}
```

TIME STEP SIZE CHANGE ENCOUNTERED (2)

The message of Part III. It does not say “unstable”; it says the integrator wanted a different step from last time. The step is `DT` until the fluid’s CFL limit `cfl` $\Delta x/|u|_{\max}$ (`CFL_MAX`, usually 0.3) falls below it; the first step at which that happens differs from the previous one, and the default is to stop rather than carry on with a step the structure’s stiffness was not checked against.

Where it came up, and what it meant each time:

- IB/01 with `DT = 1e-3` (chapter 5): the step was too large for the band’s springs; the velocity blew up within a few steps. Fix: a smaller `DT`;
- the pump’s untapered target wave (9b): one point driven 4 cm from its free neighbour; the fluid between them reached 12 m/s. Fix: taper the wave;
- the pump’s walls meeting and crossing (9b, 14’s stiffness wave): a geometry failure first, a velocity failure second. Fix: keep the walls apart;
- a bell or beam far too stiff for the grid (10, 13): springs or beams with a natural frequency above $1/\Delta t$.

Setting `warn_on_dt_change` lets a run continue with a smaller step; that hides a stiffness problem but is right for a genuinely fast transient (a start-up spike, chapter 14).

THE 2014 RULES (1): START AT REST, AND KEEP THE STIFFNESS DOWN

"Start at zero velocity. If your initial velocity is not zero, then this is like instantaneously adding a structure to a flow. Large oscillations and instabilities can occur. It is always good to ramp up the velocity of the background fluid, and sometimes even the structure." (2014, slide 2)

Where the series does it: the `tanh` ramp of the upstream cylinder's inflow (chapter 15), the raised-cosine ramp of chapter 9a's target points, the half-period ramp of the pump (9b), the gravity ramp `1 + (F-1)exp(-400t) - F exp(-600t)` of the filament (15); and where it did not: the oscillating cylinder's first step, a force spike of 523 N/m (14), and the eel's jump to 0.26 m/s (14). Both survived because the constraint method has no springs to break; a spring-and-beam structure started that way is what the 2014 slide warns about.

"Make sure your stiffness isn't too high. Relatively large values of kbend or ks can result in high frequency oscillations. This results in oscillating flow fields that fluctuate from close to zero to large magnitude. This effect is not always obvious. Plot velocity vectors of constant magnitude in time! This is often corrected by decreasing the stiffness and/or increasing the damping." (slide 3)

The numbers behind it (chapters 8b, 10, 13): a spring of stiffness K on a point whose share of fluid mass is m oscillates at $\omega = \sqrt{K/m}$, and forward Euler needs $\Delta t < 2/\omega$; the penalty copies of chapter 13 give the same rule as $\Delta t < 2\sqrt{m/K}$. Stiffness that is right for the physics can still be wrong for the step: then lower `DT`, not the stiffness, if the dimensionless stiffness (8b) is the one you want. "Velocity vectors of constant magnitude" means a VisIt vector plot with scaling off (chapter 26): a flickering field at every step is the signature.

THE 2014 RULES (2): REFINE WHERE THE VORTICES ARE

"Vortices do not form. IBAMR refines the grid near the boundaries automatically. With a 4:1 refinement ratio, this may not resolve vortices far from the boundary. For example, the default does not always generate chamber vortices that should be there. The input files contain two ways to refine the grid around vortices. You can set it to a percentage of the max vorticity. You can set an absolute value for refinement." (slides 4–5)

```
// the 2014 slide, as shown (with MATLAB-style % comments)
vorticity_abs_thresh = 0.5;
%If vorticity > 0.5, then the grid is refined.
vorticity_rel_thresh = 0.1;
%If vorticity > 10% of the max, then the grid is refined.
```

"I prefer to use the absolute threshold. The value needed dependent on the application. I suggest looking at the visit plot, determining the magnitude of vortices away from the boundary, and setting the threshold to 1/10- 1/2 of that."

THE 2014 RULES (2): REFINE WHERE THE VORTICES ARE (2)

```
// 0.19.0: both keys exist, in the Navier-Stokes integrator's block,
// as one value per refinement level (or one value for all)
INSStaggeredHierarchyIntegrator {
   using_vorticity_tagging = VORTICITY_TAGGING   // TRUE
   vorticity_rel_thresh    = 0.25               // of the maximum |omega|, each level
   // vorticity_abs_thresh = 0.5                // or an absolute value
   tag_buffer              = TAG_BUFFER
   ...
}
```

Checked in `INSHierarchyIntegrator.cpp`: `vorticity_rel_thresh` and `vorticity_abs_thresh` are read with `getDoubleArray`, one entry per level that may be refined (a single number applies to all), and `using_vorticity_tagging` must be `TRUE`. The 2014 snippet's `%` comments and semicolons are MATLAB's, not the input file's (`//`, no semicolons); pasted as shown they are a parse error (chapter 3). The upstream cylinder (chapter 15) ships `vorticity_rel_thresh = 0.25`; the series' examples mostly tag by the structure alone, which is the "default" the slide means, and chapter 7's `tag_buffer` widens the band around it.

THE 2014 RULES (3): TIGHTEN THE FLUID SOLVER

"Reducing error tolerance in fluid solver. Often this flag is needed for convergence for our problems. It sets a maximum error tolerance in the fluid solver. (The smaller it is, the longer the sims take)." (slide 6)

```
make main2d
./main2d input2d -stokes_ksp_rtol 1.0e-9
# or
make main3d
./main3d input3d -stokes_ksp_rtol 1.0e-9
```

Checked: the staggered-grid integrator still hands its Stokes solver the PETSc options prefix `stokes_` (`INSStaggeredHierarchyIntegrator.cpp`, `getStokesSolver`), so `-stokes_ksp_rtol` on the command line still reaches a PETSc Krylov Stokes solver. The input-file way, which survives a restart and is what the examples do, is a sub-block:

THE 2014 RULES (3): TIGHTEN THE FLUID SOLVER (2)

```
INSStaggeredHierarchyIntegrator {
   ...
   stokes_solver_type  = "PETSC_KRYLOV_SOLVER"   // the examples leave the type unset (the manager picks one)
   stokes_solver_db {
      rel_residual_tol = 1.0e-9                   // the 2014 flag, as a key
      abs_residual_tol = 1.0e-12
      max_iterations   = 100
   }
   stokes_precond_type = "PROJECTION_PRECONDITIONER"
   ...
}
```

Keys read by `PETScKrylovLinearSolver` (ibtk): `rel_residual_tol`, `abs_residual_tol`, `max_iterations`; the examples' `VelocityHypreSolver`, `PressureHypreSolver`, `VelocityFACSolver` and `PressureFACSolver` blocks (chapter 7) set the sub-solvers the same way. When to tighten: a drift in a conserved quantity (the fluid momentum of chapter 14's example 02, the band's area of slide 27), a pressure field with a checkerboard, or a convergence study that stalls. Each decade of tolerance costs iterations, not accuracy of the discretization; it buys back only what the solver was losing.

THE 2014 RULES (4): MATCH EVERY DIMENSIONLESS NUMBER, NOT ONLY RE

"More than matching Re. As mathematicians, we may not think about real units very much. You can match Re, but is that good enough? Not always, there are other dimensionless numbers to match: Dimensionless bending and stretching stiffness; Strouhal number (flapping frequency, pumping frequency)." (slide 7) "Be safe: match all numbers to reality. Sometimes its easiest to construct geometries and velocities around unity. If you just match them to reality, you don't have to worry about matching ever important quantity. If you have a lot of results that are not matched to reality, then you have to make things dimensionless and compare them to the real thing." (slide 8)

"The right frequency. Let's say you are looking at oscillating flows or flapping. You calculate Re based on max velocity. What is the right flapping or pulsing frequency? Use dimensionless time – $t' = tU/L$. Calculate the correct dimensional time based on the application. Determine what your dimensional time should be to match that value." (slide 10)

THE 2014 RULES (4): MATCH EVERY DIMENSIONLESS NUMBER, NOT ONLY RE (2)

"Strouhal number. Matching dimensionless time is related to matching the Strouhal number. St = f U/L, where f is the frequency of vortex shedding. In many biological applications, the frequency of vortex shedding equals the frequency of flapping (vortex shed during each direction change)." (slide 11)

"Dimensionless bending. By using the dimensionless beam stiffness k'_{beam}, one can make an educated guess for k_{beam}. Setting $k'_{\mathrm{beam}} = 1$ typically results in a little bit of bending (maybe 2 - 10 degrees). In 2D, the relationship for Re > 1 is: $k'_{\mathrm{beam}} = k_{\mathrm{beam}}/(\rho U^2 L^4)$. . . . Warning to user: these are really rough estimates." (slides 12–13)

Two corrections. The Strouhal number is $\mathrm{St} = fL/U$, not fU/L (chapter 8b; the 2014 slide inverted it). In 2D, with k_{beam} per unit depth, the dimensionless bending stiffness is $k_{\mathrm{beam}}/(\rho U^2 L^3)$, the form of chapter 8b and of the 2020 non-dimensionalization deck; L^4 is the 3D form. Chapter 10 measured what the rule of thumb is worth: the jellyfish bell at $k'_{\mathrm{beam}} = 10^{-3}$ did nothing, at 1 it swam.

THE 2014 “OTHER POINTS COMING SOON”, WRITTEN NOW

The 2014 deck ended: “Other points coming soon: boundaries touching; violating incompressibility with boundary conditions; how to minimize oscillations; boundary leaves domain – may or may not crash.” They never came. From Part III:

- Boundaries touching. The IB method has no contact handling: two walls driven into the same grid cells share the fluid velocity there and stick (chapter 9b’s pump, five times; 14’s stiffness wave). Keep them apart by design (a gentler force, a longer rest length, a taper), stiffen what collapses, or add a contact force of your own (chapter 9b’s other laws). Once stuck, nothing but a prescribed motion pulls them apart (9b’s target wave reopened; its force wave did not).
- Violating incompressibility with boundary conditions. An inflow with no outflow in a box with walls, or a prescribed velocity on every side whose net flux is not zero, has no solution: the pressure solver fails to converge or the velocity field does something strange near the walls (chapter 7’s rules: inflow against outflow, or periodic; `normalize_pressure` only for all-periodic or all-wall boxes).
- Minimizing oscillations. Ramp the start (slide 8); match the stiffness to the step; damping on target points (chapter 9a’s `uniform_target_damping`); do not resolve a structure finer than the grid (points $\Delta x/2$ apart, chapter 5); and read the oscillation first: a flicker at every step is numerical, a wobble over many steps is physics (chapter 10’s curling beam overshot at low viscosity and that was right).
- Boundary leaves the domain. In a periodic direction a point wraps round and nothing happens. In a direction with physical boundaries `LDataManager` has a check that stops the run with `IB point has escaped from the computational domain!`, but `IBMethod` leaves it *off* unless the input says `error_if_points_leave_domain = TRUE` in its block; off, the point is simply on no patch any more and the run carries on with a wrong force, slows, or stops with some other error, which is exactly “may or may not crash”. Turn the check on while developing; chapter 13’s spider got a box tall enough for its fall for this reason.

THE SILENT FAILURES

Not every failure prints. These were found in Part III by their effects:

- A key in the wrong block, or misspelt. Ignored without a word: the 2020 `PrintOutput` block outside `ConstraintIBMethod` (chapter 14), a `uniform_target_stiffness` under the wrong structure name (9a), `N` changed in the input but not in the generator (8a). Check the output folder for what you asked for, and the log's echo of the input database (`input_db->printClassData(plog)` in every example's `main`).
- A generator and an input that disagree. Point spacing, the structure's index ranges, the box size: chapter 8a's rule, change one thing and rerun the generator; the pump's `pump_geometry.h` and the posts' README (15) exist because of it.
- A spring force function index with no function registered (9b): the standard force generator dereferences a null pointer and the run dies with a segmentation fault at the first force evaluation, no message.
- `viz_writer = "VisIt"` without `"Silo"` in a program that registers a Silo writer (13): a segmentation fault at the first visualization dump, after the run has started correctly.
- A structure on the wrong level (14): "`Structure X does not exist on level N`" from `ConstraintIBKinematics`; the standard initializer places structures by `level_number`, and chapter 9a's update functions look on one level too.
- Several MPI jobs on one machine (chapter 2): Open MPI binds every job to the same cores and all of them crawl; `mpirun --bind-to none`.

INPUT FILES AND BUILDS: WHAT CHAPTER 3 FOUND, IN ONE PLACE

- Windows line endings in an input file make the parser see every line as unterminated: save with LF (chapter 3's recipe), or the 2012 trick of ending every line with `//`.
- Curly quotes pasted from a slide (`"0.0“`, chapter 14) and MATLAB `%` comments or semicolons (slide 9) are parse errors with a line number; a missing closing brace is reported at the end of the file.
- A muParser expression it cannot read stops the run at the first time step (chapter 3's example): no `if()`, the ternary `?:` instead (chapter 14's impulse); variables are `X_0`, `X_1`, `t`; undefined names are errors at the first evaluation, not at parse time.
- Build errors: the upstream example's own `CMakeLists.txt` outside IBAMR's tree, helper programs with a second `main`, a libMesh example on an IBAMR built without it (chapter 15); the autotools Makefiles of the old decks against an autoibamr install (chapter 3).
- Running from the wrong folder: inputs and structure files are read relative to the working directory, so `./build/main2d input2d` from the example folder, never from `build/`.

ON THE CLUSTER: THE 2019 DEBUGGING TIPS, FOR PUMA

The 2019 *Debugging tips* (four slides, UNC's Dogwood, repeated in the 2020 Dogwood deck) covered three complaints:

- "The number of nodes requested can freeze your job": a request the queue cannot satisfy sits forever with a QOS message. On Puma: ask for one node and `--ntasks` ranks within it for every example of this series; `squeue -u $USER` shows the reason column (chapter 1).
- "My job randomly stopped": read the `.out` file and `IB2d.log`; the usual causes are the time limit (`--time`), memory (`--mem-per-cpu`, 4 GB each in this series' scripts; an 8-rank 3D run can need more), and a disk quota filled by viz dumps (chapters 9b, 14, 15). The 2019 slide's "PETSc errors mean too little memory" is one case of several: a PETSc error is also a solver that did not converge, which is slide 11.
- "My job is super slow": too many ranks for too few patches (each rank wants at least a few patches of `smallest_patch_size`), a `DT` far below what the CFL allows, or viz every step. The slide's "use power-of-2 process counts" is a habit, not a rule.

ON THE CLUSTER: THE 2019 DEBUGGING TIPS, FOR PUMA (2)

What to check before you submit, in order:

1. the smoke test: `END_TIME` of 40 steps on the login node or at home (every example of this series was checked that way; chapter 3);
2. the counts in the log: vertices, springs, beams, targets as the initializer reports them, against the generator;
3. `DT` against the stiffness (slide 8) and the dump intervals against the run length;
4. the job script's `--ntasks`, `--time`, `--mem-per-cpu` against the measured cost of a similar run (the README of each example gives one);
5. then submit, and `grep "dt = "` the log after the first minute.

The 2019 slides conflated nodes with MPI tasks ("nodes requested" when tasks were meant); on Slurm, `--nodes` is machines and `--ntasks` ranks, and this series' scripts always use one of the former.

STABILITY, CONVERGENCE AND ORDER: THE 2021 LECTURE'S DEFINITIONS

"Numerical analysis is not only the design of numerical methods, but also their analysis. Three central concepts in this analysis are: convergence: whether the method approximates the solution, order: how well it approximates the solution, and stability: whether errors are damped out." (MATH 481 Lecture 19, 2021, slide 3)

"Numerical instability. Typically, algorithms would approach the right solution in the limit, if there were no round-off or truncation errors, but depending on the specific computational method, errors can be magnified, instead of damped, causing the error to grow exponentially." (slide 4)

"A numerical method is said to be convergent if the numerical solution approaches the exact solution as the step size h goes to 0. In the immersed boundary method you typically need to worry about convergence in space. Is the flow around the boundary well resolved? Are shed vortices well resolved?" (slide 19)

STABILITY, CONVERGENCE AND ORDER: THE 2021 LECTURE'S DEFINITIONS (2)

"Rate of convergence. In numerical analysis, the speed at which a convergent sequence approaches its limit is called the rate of convergence. ... The sequence converges with order q to L for $q > 1$ if" $|x_{k+1} - L| \leq C\,|x_k - L|^q$; "if it converges with order $q = 2$, it is called quadratic convergence, $q = 3$, it is called cubic convergence." (slides 28–29)

For a discretization the usual statement is in the grid spacing: the error is Ch^p and p is the *order*; halving h divides the error by 2^p, so from two grids $p = \log_2(e_h/e_{h/2})$. Peskin's IB method is second order in space for smooth problems and first order where the structure has an edge or a kink (chapter 5; the 2021 slide 33 says the same of IB2d), and first or second order in time by version. The CFL condition: "a necessary condition for stability while solving certain partial differential equations (usually hyperbolic PDEs) numerically by the method of finite differences ... Typically $C_{\max} = 1$. In immersed boundary simulations, we rarely have to worry about this." (slide 18) For the *fluid* that is so at the speeds of Part III; the limit that bites is the stiffness, and IBAMR's `cfl` key and the abort of slide 6 are nonetheless a CFL check.

TWO EXERCISES YOU CAN RUN IN A MINUTE: A STIFF EQUATION AND THE HEAT EQUATION

"Euler's Method. $y'(t) = -15y(t) = f(t, y(t))$, $y(0) = 1$. Let $h = 1/4$. $y_1 = y_0 + hf(t_0, y_0) = 1 + (1/4)(-15) = -2.75$; $y_2 = -2.75 + (1/4)(-15)(-2.75) = 7.5625$." (slide 6) "Open stiff_ode_example.m . . . Try the following parameter values to observe the instability: stiff_ode_example(0.2, 1), stiff_ode_example(0.1, 1), stiff_ode_example(0.01, 1)" (slide 13)

```
# examples/support/matlab-exercises (MATLAB/Octave and Python versions)
python3 stiff_ode_example.py 0.2 1     # grows: dt > 2/15
python3 stiff_ode_example.py 0.1 1     # oscillates in sign: 1/15 < dt < 2/15
python3 stiff_ode_example.py 0.01 1    # decays like exp(-15 t)
```

Euler on $y' = -\lambda y$ is stable for $\Delta t < 2/\lambda$: the same inequality as a spring's $\Delta t < 2\sqrt{m/K}$ (slide 8) with $\lambda = \sqrt{K/m}$. The 2021 script's `zeros(N)` made $N \times N$ arrays and stopped one step short of `tfinal`; the shipped versions fix both.

TWO EXERCISES YOU CAN RUN IN A MINUTE: A STIFF EQUATION AND THE HEAT EQUATION (2)

"Heat equation. To look at an example that incorporates issues of instability and convergence in space and time, we will simulate the heat equation. . . . This script implement a finite difference scheme for the heat equation with $D = 1$. Both analytical and numerical solutions are plotted at a single time ($t = 0.1$)." (slides 20–21) "Try the following parameters to show convergence in space: Set $T = 100$. Try $n = 3, 5, 10, 20$." (slide 25) "Now we will consider instability due to too large of a time step, as well as convergence in time. Set $n = 10$. Try $T = 2, 4, 8, 16, 32, 64$. Since the simulation runs until time $= 0.1$, $dt = 0.1/T$." (slide 26)

```
python3 heat_equation.py 10 2      # r = dt/dx^2 > 1/2: blows up
python3 heat_equation.py 10 64     # stable, converged in time
python3 heat_equation.py 20 100    # finer in space
```

The explicit scheme needs $r = \Delta t/\Delta x^2 \leq 1/2$: refining in space without refining in time breaks it, which is why IBAMR's inputs write `DT` in terms of `DX_FINEST` (chapter 15's examples: `DT = 1.6e-2*DX_FINEST`, `0.128/NFINEST^2`). The script's header said `sin(3*pi*x)` where the code uses `sin(2*pi*x)`; fixed in the shipped version.

THE 2021 LAB IN IB2D, TRANSLATED TO IBAMR

"Immersed boundary method. Typically you want 2 boundary points for every fluid grid point, dx/2 = ds. Second order in space if smooth boundaries. First order in space if edges. For publication quality results, typically use 256x256 or 512x512 grids for fluid domain. Time step either first or second order, depending on version. Typically use dt = 10^{-5} or 10^{-6}, when characteristic time is t = 1 when grid is 512x512." (slide 33)

The lab (slides 34–39): set the rubber band's `dt` to 1.0 and `Tfinal` to 100: "Instability error! `BLOW UP! (*forces TOO large*) -> try decreasing the time-step or decreasing material property Stiffnesses`"; put `dt` back, raise `kappa_spring` from `2.0e3` to `2.0e6`: the same error ("There is an inverse relationship between the force the boundary applies to the fluid and the time step required"); then `N` from 32 to 64 in the generator with `npts = ceil(2*(band_length/L)*N)` ("This will set ds to approximately dx/2") and `Nx`, `Ny` in the input.

THE 2021 LAB IN IB2D, TRANSLATED TO IBAMR (2)

IB2d (2021 lab)	IBAMR (this series)
`dt = 1.0`, `Tfinal = 100`	`DT = 1e-3` in IB/01: “Time step size change encountered” within a few steps (chapter 5)
`BLOW UP! (*forces TOO large*)`	no such message: the velocity grows until the CFL check stops the run (slide 6), or a point escapes (slide 15)
`kappa_spring = 2.0e6`	`kappa_spring` in IB/01’s generator; chapter 9a’s tether stiffness; the same inverse relation
`N = 32` to 64, `npts = ceil(2*(band_length/L)*N)`	`N` in the generator and `N` in the input; the files `curve2d_64` to `_512` of the upstream ex1 are this done four times (slide 26)
`Nx = Ny = 64` in `input2d`	`N`, `MAX_LEVELS`, `REF_RATIO`: `NFINEST` is the grid that must match the generator (chapter 7)
256^2 or 512^2 for publication	the same, with AMR levels making the 512 affordable (chapters 5, 7)

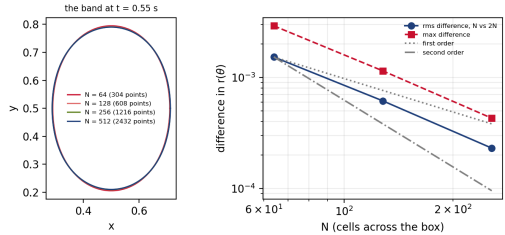
The lab’s third exercise is a convergence step without the comparison; the next two slides do the comparison.

A CONVERGENCE STUDY (1): THE BOUNCING BAND AT FOUR GRIDS

The upstream `IB/explicit/ex1` (chapter 15) ships its band at four resolutions, `curve2d_64` to `curve2d_512`, generated with `ds = dx/2` for $N = 64, 128, 256$ and 512 cells across the unit box. The input's `DT = 1.6e-2*DX_FINEST` halves with the cell, and `END_TIME = 0.55` s is fixed, so the four runs are the same problem at four grids and four steps at once, the way a refinement study should be done when the step is tied to the grid by stability.

Run for this chapter with `N` and the structure name changed in the input and the positions dumped (`data_dump_interval`), four ranks each, all four at once:

N	points	steps	Δt	wall time, 4 ranks
64	304	2,200	2.5×10^{-4}	28 s
128	608	4,400	1.25×10^{-4}	78 s
256	1,216	8,800	6.25×10^{-5}	303 s
512	2,432	17,600	3.125×10^{-5}	19 min



Left: the band at $t = 0.55$ s on the four grids. Right: the rms and maximum difference of the band's radius as a function of angle between each grid and the next finer one, against first- and second-order lines.

IBAMR 0.19.0, `examples/IB/explicit/ex1`, inputs as shipped except `N`, the structure name and the position dumps; `tools/ch16_conv.py`.

A CONVERGENCE STUDY (2): WHAT THE NUMBERS SAY

The band starts as an ellipse of area A and, incompressible fluid inside and out, must relax toward a circle of radius $\sqrt{A/\pi}$ while oscillating; at $t = 0.55$ s it is still oscillating, so the four shapes are compared with each other, not with the circle.

grids	rms difference of $r(\theta)$	max difference	mean radius	$(r_{\max} - r_{\min})/\bar{r}$
64	–	–	0.2527	0.336
$64 \to 128$	1.53×10^{-3}	2.92×10^{-3}	0.2523	0.320
$128 \to 256$	6.16×10^{-4}	1.15×10^{-3}	0.2521	0.313
$256 \to 512$	2.31×10^{-4}	4.30×10^{-4}	0.2520	0.311
observed order (rms; max)	1.31, then 1.41	1.35, then 1.42		

The band's radius as a function of angle moves by 1.5 mm when the grid goes from 64 to 128 cells and by 0.6 mm from 128 to 256 (on a band of radius 0.25 m in a 1 m box): the differences shrink by a factor 2.5 per halving, an observed order of about 1.3, between first and second. That is what the IB method does for a thin elastic interface: the velocity is continuous across the band but its derivative jumps, and the regularized delta function smears the jump over a few cells, so the error is first order there and second order away from it (chapter 5; Griffith and Peskin 2005). The mean radius converges to 0.2521 (0.1 per cent between the first two grids, 0.04 between the next); the circle of radius 0.25 it is heading for is not yet reached because the band is still oscillating at 0.55 s ($r_{\max} - r_{\min}$ is a third of the radius on every grid). The 512 grid confirms it: the difference falls to 0.23 mm and the order rises to 1.4; the mean radius settles at 0.2520, with the 64 grid 2.7 per cent and the 256 grid 0.1 per cent of the radius away from it in the rms sense.

A CONVERGENCE STUDY (3): THREE STUDIES ALREADY IN THE TREE

For when you want the official version:

- `adv_diff/ex0/converg2d`: a text file of results, not a script, from an advected–diffused lattice of Gaussians at $N = 16$ to 256 against the exact solution: "convergence rates: L1-norm: 1.9949 1.9989 1.9998 2.0000; L2-norm: 2.0000 ..." for $\kappa = 0.1$, and still 1.99 at $\kappa = 0.01$: the second-order scalar solver of chapter 24;
- `navier_stokes/ex1/convergence_tester.cpp`: a separate program that reads the hierarchy dumps of a coarse and a fine run of the same exact-solution flow (`hier_dump_interval`, or `coarse_hier_dump_interval` and `fine_hier_dump_interval` in `Main`) and prints the norms of their difference;
- IB/01's time-step study (chapter 5, the README): the band's shape the same to four digits for `DT` from 10^{-6} to 5×10^{-4}, and the abort at 10^{-3}: convergence in time has a cliff, not a slope, once the step is stiffness-limited.

HOW TO DO YOUR OWN CONVERGENCE STUDY

1. Fix the problem. Choose the dimensionless numbers (chapter 8b) and keep them: the same Re, k', St on every grid. Change nothing but the resolution.
2. Refine together. Halve Δx (`N`, or one more `MAX_LEVELS`), regenerate the structure with $\Delta s = \Delta x/2$ (the generator's `N`), and halve `DT` if it is tied to the grid (it is, in every input of this series); three grids are the minimum for an order, four are better.
3. Pick a number. Something the physics cares about and the output gives: a drag coefficient (chapter 11), a swimming speed (14), a radius or an amplitude (this chapter), the transport per period (9b). Not a picture.
4. Compare neighbours. $e_N = |q_N - q_{2N}|$ between successive grids, or against the finest; the order is $p = \log_2(e_N/e_{2N})$. Expect about 2 for smooth structures and 1 where there are kinks, corners or thin boundaries with jumps across them (chapter 5), and less in time if the step is stiffness-limited.
5. Know when to stop. When e_N is below what you will claim, and when the next grid costs more than the question is worth: the 512 grid of this study cost 19 minutes against 28 s at 64 (the four ran at once on 16 ranks of one desktop PC, so the ratio, 40, is the number that matters: eight times the cells and eight times the steps); in 3D each halving is sixteen times the cost (chapter 12).
6. Tighten the solver first if the differences stall at a floor (slide 11), and check the dump interval: compare at the same *time*, not the same step.

A DEBUGGING ROUTINE

When a run misbehaves, in this order; most problems are found before step 5.

1. Read the end of the log and the screen output: an `Error message` block names the class and the reason (slides 31, 32); no message plus a segmentation fault means a null pointer or a missing writer (slide 16).
2. Shorten the run to 40 steps with viz every step and look at the first frames in VisIt (chapter 26): a structure that explodes at step 1 is a file or stiffness problem; one that drifts for hundreds of steps and then goes is a boundary-condition or contact problem.
3. Halve `DT`. If the failure moves later or vanishes, it was the step. Then decide whether the stiffness or the step was wrong (slide 8).
4. Remove things until it works: one structure at a time, the forcing function, the update function, the boundary conditions (all periodic), the extra levels (`MAX_LEVELS = 1`). What you put back last is the cause.
5. Count. Vertices, springs, beams, targets in the log against the generator; the structure order in `structure_names` against the indices the update functions use (chapters 8a, 9a, 9b).
6. Run in serial (`-np 1`): a failure that depends on the number of ranks is a point crossing a patch boundary or a parallel-only code path.
7. Compare with the shipped example it was copied from: run that, unchanged, and diff the inputs.
8. Then the tools: a debug build (`-DCMAKE_BUILD_TYPE=Debug`, or autoibamr's `--prefix=~/autoibamr-debug` of chapter 2) under `gdb --args ./build/main2d input2d` gives a backtrace at the fault; PETSc's `-ksp_monitor` and `-log_view` show a solver that is struggling; `valgrind` finds the uninitialized value.

WHAT CAN GO WRONG (1): THE MESSAGES, COLLECTED

message	cause	chapter
`Time step size change encountered. Aborting.`	the CFL-limited step fell below `DT`: an instability, a tearing structure, or a real fast transient	5, 9b, 10, 13, 14; slide 6
`IB point has escaped from the computational domain!`	a point crossed a non-periodic boundary, with `error_if_points_leave_domain = TRUE` in the `IBMethod` block (off by default)	13; slide 15
`Structure X does not exist on level N`	`structure_levels` and `level_number` disagree, or a misspelt name	14
`Update methods supported are CONSTRAINT_VELOCITY, ...`	a bad `lag_position_update_method`	14
`frequency of cylinder does not exist in the InputDatabase`	a kinematics class's required key missing	14
`incomplete beam curvature specification`	a `.beam` line with some but not `NDIM` curvature columns	10
`muParserRobinBcCoefs:: setDataOnPatch()` error at step 1	an expression muParser cannot read	3, 7
a parse error with a line number	CRLF endings, curly quotes, `%` comments, a missing brace	3, 14; slide 17
`Example ... requires libMesh - skipping`	an IBFE or IIM example on an IBAMR without libMesh	15
`physical domain must be a single box`	`domain_boxes` with more than one box	7

WHAT CAN GO WRONG (2): THE SYMPTOMS WITHOUT A MESSAGE

what you see	cause	chapter
a segmentation fault at the first force evaluation	a spring force function index with no function registered	9b
a segmentation fault at the first viz dump	`viz_writer` without `"Silo"` in a program that registers the Silo writer	13
nothing printed where output was expected	a `PrintOutput` or output key in the wrong block, or misspelt	14
walls that meet and never reopen	no contact handling: points in one cell share a velocity	9b, 14
a structure that does nothing	stiffness far too small for the grid and flow ($k'_{\mathrm{beam}} \ll 1$)	10
a flickering velocity field	stiffness too high for the step	9a, 10; slide 8
all runs ten times slower than yesterday	several `mpirun` jobs bound to the same cores	2
the disk full	viz or restart dumps every few steps for a long run	9b, 14, 15
a Puma job that never starts, or stops early	an unsatisfiable request; the time, memory or quota limit	1; slide 18
the band drifts from its generator's shape at step 0	the generator's `N` or `ds` and the input's grid disagree	8a

"What can go wrong" slides with the same table in short form: chapters 5, 6, 7, 8a, 8b, 9a, 9b, 10, 11, 12, 13, 14 and 15.

HOMEWORK

1. IB/01: raise `DT` until the run aborts, then set `warn_on_dt_change = TRUE` and `error_on_dt_change = FALSE` and run again; plot `dt` from the log against time and say what the integrator did instead of stopping.
2. Run the two Python exercises of slides 22–23 at the lecture's parameters and at one more of your own each; write the stability condition you observed as an inequality.
3. Repeat the convergence study of slides 26–27 on IB/03 (chapter 8a's tethered beam) with the drag coefficient of chapter 11 as the number: three grids, one order.
4. Take the upstream cylinder of chapter 15 (`IB/explicit/ex2`) and set `vorticity_rel_thresh` to 0.5, 0.25 and 0.1; count the finest-level cells in the log at $t = 20$ s and compare the wake in VisIt.
5. Make a run fail on purpose in three different ways (a key in the wrong block, a point outside the box, a stiffness ten times too high) and write down, for each, the first line of the log that gave it away.

RULES OF THUMB

- `grep "dt = " IB2d.log` first; `integral{F}` second.
- “Time step size change” means the fluid got faster than `DT` allows; find out why before loosening the check.
- Ramp every start; match the step to the stiffness, not the stiffness to the step.
- Refine where the vortices are: `vorticity_rel_thresh` or `_abs_thresh`, `tag_buffer`.
- Tighten the solver when a conserved quantity drifts; it costs iterations, not accuracy.
- Match Re, k', St, Wo together, and $\mathrm{St} = fL/U$.
- Walls do not touch, points do not leave, keys sit in the right block.
- Three grids, one number, one order; the step refined with the grid.
- Shorten, halve, remove, count, serialize, compare, then `gdb`.

SUMMARY

1. IBAMR's loudest failure, the time-step-change abort, is a CFL check on a fixed step; its usual causes in Part III were a step too large for a stiffness, a structure driven into itself or into a step, and a real transient. Watching `dt` in the log, as the 2012 ReadMe said, still catches it earliest.
2. The 2014 rules stand, with their keys verified: ramp the start, keep the stiffness and the step consistent, refine by vorticity thresholds in the integrator's block, tighten the Stokes solver by its sub-block (or the old `-stokes_ksp_rtol`), and match every dimensionless number; the Strouhal number and the 2D bending stiffness are corrected.
3. The promised "other points" are written: walls touching stick, a net inflow has no solution, oscillations have numerical and physical kinds, and a point leaving the box is caught by a named message only if the check is switched on.
4. Convergence means the number you care about stops changing as the grid and the step are refined together; the bouncing band gives an observed order of 1.3 to 1.4 across the 64, 128, 256 and 512 grids, IBAMR's own scalar study gives second order, and the time-step study of IB/01 shows a cliff.
5. When a run misbehaves: read, shorten, halve, remove, count, serialize, compare, debug, in that order.

This closes Part III. Part IV, chapters 17 to 25, is the finite-element method, IBFE.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The 2014 deck's five rules are quoted and each checked against 0.19.0: both vorticity keys exist (with the block and the per-level form stated), the `stokes_` prefix survives and the input-file form is given, the Strouhal number and the 2D bending exponent are corrected, and the MATLAB-style snippet is marked as unparseable.
- Its "other points coming soon" are written from the series' own runs.
- The 2021 lecture's definitions and exercises are kept, its IB2d lab is mapped to IBAMR, and its claim that the CFL condition rarely matters in IB is qualified with where IBAMR applies one.
- The convergence study is new: four grids of an upstream example with measured differences and an order; the three studies already in the tree are pointed out.
- The 2019 cluster tips are translated to Puma and their two inaccuracies named; the debugging routine and the two tables are new, with every message traced to the source or to a chapter.

RECORDINGS (LISTED, NOT INCORPORATED)

The 2014 *What can go wrong* tutorial has no recording. The 2021 MATH 481 Lecture 19 was given in class with a VCAT (remote desktop) session for the MATLAB exercises; no recording is in the course's folders. Nothing from a recording is reproduced here.

REFERENCES

- IBAMR 0.19.0: `src/IB/IBHierarchyIntegrator.cpp` (the dt-change check and keys), `src/navier_stokes/INSHierarchyIntegrator.cpp` (vorticity thresholds), `src/navier_stokes/INSStaggeredHierarchyIntegrator.cpp` (the Stokes solver and its prefix), `ibtk/src/lagrangian/LDataManager.cpp` (the escaped point), `ibtk/src/solvers/impls/PETScKrylovLinearSolver.cpp` (tolerance keys), `examples/IB/explicit/ex1`, `examples/adv_diff/ex0/converg2d`, `examples/navier_stokes/ex1/convergence_tester.cpp`. https://github.com/IBAMR/IBAMR
- C. S. Peskin, *The immersed boundary method*, Acta Numerica 11 (2002) 479–517 (accuracy of the method); B. E. Griffith and C. S. Peskin, *On the order of accuracy of the immersed boundary method: higher order convergence rates for sufficiently smooth problems*, J. Comput. Phys. 208 (2005) 75–105.
- This series: `examples/support/matlab-exercises` (`stiff_ode_example`, `heat_equation`, MATLAB and Python), `tools/ch16_conv.py`, IB/01's README (the time-step table), chapter 3's error table.
- Sources: *11-IBAMR Tutorial: What can go wrong* (2014, 14 slides); *MATH 481 Lecture 19: Stability and convergence* (2021, edited 2023, 40 slides); the lab ReadMe deck (2012), slide 5; *Debugging tips* (2019, 4 slides, repeated in the 2020 Dogwood deck).